Aasmund Gabestad Nørsett

Delayed Weight Updates for High Throughput Hardware Accelerator of the CCSDS-123.0-B-2 Compression Algorithm

Master's thesis in Electronic System Design and Innovation
Supervisor: Milica Orlandic
Co-supervisor: Samuel Boyle
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Electronic Systems

NTNU

Abstract

Sammendrag

AI Declaration

Contents

Abstract . . . . . **i**
Sammendrag . . . . . **ii**
AI Declaration . . . . . **iii**
Contents . . . . . **iv**
Acronyms . . . . . **v**
1 Introduction **1**
2 Background **2**
2.1 Hyperspectral Image Compression 2
2.1.1 Predictor 5
2.1.2 Header 6
2.1.3 Entropy Encoder 6
2.2 Hardware Accelerators 7
2.3 Coding Performance and Decompressed Image Quality 7
3 State-of-the-Art **9**
4 Previous Work **11**
5 Implementation and Verification **14**
5.1 Verification Model 14
5.2 Hardware Modifications 14
5.3 Hardware Verification 14
5.4 Hardware Testing 14
6 Results **18**
6.1 Verification Model 18
6.2 Compression Performance 18
6.3 RTL Results 19
6.4 Hardware Testing 19
7 Discussion and Further Work **21**
8 Conclusions **22**
Bibliography **23**
A Block Design **25**

Acronyms

BI Band Interleaved. 3

BIL Band Interleaved by Line. 3, 4, 11

BIP Band Interleaved by Pixel. 3, 4, 11

BSQ Band Sequential. 3

CCSDS Consultative Committee for Space Data Systems. 2

CLB Configurable Logic Blocks. 8

DPCM Differential Pulse Code Modulation. 2

DSP Digital Signal Processor. 8

FL Fast Lossless. 2

FLEX Fast Lossless Extended. 2

FPGA Field-Programmable Gate Arrays. 1, 2, 8

HYPSO HYPerspectral Smallsat for Ocean observation. 1, 2

NTNU Norwegian University of Science and Technology. 1

Chapter 1

Introduction

With the development of small satellite technologies such as cube satellites (cubesats), with lower costs and shorter development times, the availability of remote sensing data has increased. Hereunder, hyperspectral and multispectral imagers provide data that is vital for ocean monitoring, geological research and observation of other natural processes [1, 2].

As part of the HYPerspectral Smallsat for Ocean observation (HYPSO) project at the Norwegian University of Science and Technology (NTNU), two 6U cubesats, HYPSO-1 and HYPSO-2, were launched in 2022 and 2024 respectively. These carry hyperspectral imagers capable of sensing wavelengths ranging from 430 nm to 800 nm over 120 spectral bands. Their primary purpose is to monitor algal blooms in the ocean, which can give indications as to the health of marine environments [3]. This is made possible by the reflection of characteristic wavelengths from chlorophyll, detectable by the on-board hyperspectral imagers.

A nominal HYPSO image has a size of roughly 150 MB. This presents a challenge due to limited on-board resources for storing large amounts of data, and low downlink speeds. As an example, HYPSO-1 is restricted to 75 MB of downlinked data per orbital revolution [3]. This prompts the need for on-board image compression to increase the scientific output and value of the mission.

Small satellites operate under strict limitations on on-board resources such as processor time and power budgets. This, combined with the complexity of image compression algorithms, makes the use of custom hardware accelerators ideal. For their implementation, Field-Programmable Gate Arrays (FPGA) are the perfect choice, providing great flexibility and speed at a low cost.

This thesis explores the use of a novel weight update scheme proposed by Jia et al. for the CCSDS-123.0-B-2 compression algorithm. The purpose is to increase the throughput of the accelerator implemented by [5] for use on-board the HYPSO missions. This should not come at significant degradation in compression performance or hardware resource utilization. Additionally, an open-source decompressor has been implemented in order to verify the results of the proposed change in the algorithm.

The rest of the paper is organized as follows...

Chapter 2

Background

2.1 Hyperspectral Image Compression

The Consultative Committee for Space Data Systems (CCSDS) is a multi-institutional working group developing algorithms and standards for increased cooperation amongst space actors. A portion of their work has been concerned with the effective encoding of hyperspectral and multispectral images. Standard CCSDS 123.0-B-1, hereby referred to as issue 1, describes a lossless compression algorithm tailored for high throughput FPGA implementations. Issue 1 has been implemented and used successfully by several missions, the HYPSO project being one of them [1, 6]. In 2019, Issue 1 was followed by standard CCSDS 123.0-B-2, referred to as Issue 2. The following sections are based on the description of issue 2 presented in The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

Issue 1 formalizes the Fast Lossless (FL) algorithm presented by Klimesh, which uses an adaptive filtering prediction technique to achieve lossless compression at low computational complexity suitable for space applications [8, 9]. To meet the needs of limited satellite downlink capabilities, lossy compression techniques have seen increasing use [9]. For this reason, issue 2 introduces a new near-lossless feature based on the Fast Lossless Extended (FLEX) algorithm presented by Keymeulen et al. [10]. Near-lossless compression, as opposed to lossy compression, provides the user with fine-tuned control over the per-pixel loss through tunable error limit parameters.

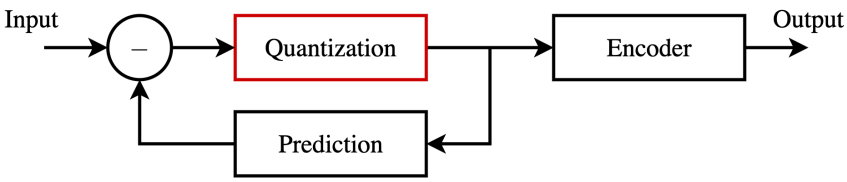

Figure 2.1: Layout of a differential pulse code modulator. The prediction loop, with optional quantization, reduces the entropy in the data passed to the encoder.

The issue 1 and issue 2 compressors use a form of Differential Pulse Code Modulation (DPCM), presented in Figure 2.1. A feedback loop, with optional quantization, predicts the value of the current input sample based on previously processed samples. The difference between the sample and the prediction, called the prediction residual, is encoded in the final bitstream. This exploits the correlation between neighbouring samples inherent in image data to reduce the entropy of the data passed to the encoder, allowing a more efficient encoding. Samples are processed in a single pass using simple operations. This reduces memory requirements and makes the algorithm suitable for hardware implementations. For the rest of this document, the prediction feedback loop will be referred to as

the predictor.

Predictor outputs are passed to an entropy encoder which losslessly encodes prediction residuals in the final bitstream. Entropy encoders, which will be covered in greater detail later in this document, encode samples with a varying number of bits based on the frequency of their occurrence in the image. A novel feature of issue 2 is the inclusion of a hybrid encoder, in addition to the sample adaptive and block adaptive encoders of issue 1. This performs better at the low encoding bitrates achievable in the near-lossless mode. After encoding, a header containing image metadata and compressor configuration parameters is prepended to the compressed bitstream (image body). This is required by the decompressor in order to correctly reconstruct the original image.

The predictor uses an adaptive linear filter method based on the work presented by Gersho [9, 11]. Observed quantities x_k, analogous to the quantized prediction residuals of Figure 2.1, which are statistically dependant on the process d_k, analogous to the image samples, produce estimates z_k of the desired samples d_k. A linear preprocessor ϕ takes previously observed quantities x_k and transforms them to a vector $\mathbf{u}_k$ given by

$$\mathbf{u}_k = \Phi(x_k, x_{k-1}, \ldots). \tag{2.1}$$

The elements of $\mathbf{u}_k$ are weighted by a vector $\mathbf{c}_k$ to produce the prediction z_k as

$$z_k = \mathbf{c}_k \cdot \mathbf{u}_k. \tag{2.2}$$

The weight vector is adapted (updated) in increments $\Delta\mathbf{c}_k$ based on observed error residuals $z_k - d_k$ of previous predictions.

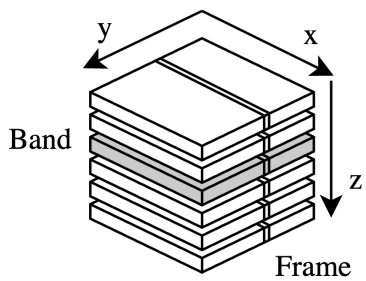

Figure 2.2: Layout of a hyperspectral image. Samples of the same z coordinates belong to the same frequency band, whereas samples of the same y coordinate are denoted as frames. Samples within the same frame of a band are denoted as lines.

Figure 2.2 shows the layout of a hyperspectral image. The sizes of the dimensions x, y, and z are denoted N_x, N_y, and N_z. Samples of the same z coordinate correspond to frequency bands, while samples of the same y coordinate are denoted as part of the same frame. In a bushbroom images, like the one on-board the HYPSO-1 mission, images are captured one frame at a time [3]. As the satellite moves, subsequent frames are concatenated to form the final image. The camera frame rate controls the time, and thus physical space, between captured frames. Samples within the same frame of a band are referred to as lines, and samples of the same x and y coordinate belong to a pixel.

In order to correctly reproduce the original image, the sample processing order of the compressor and decompressor must be the same. The standard recognizes two main categories of processor orders: Band Sequential (BSQ) and Band Interleaved (BI). BSQ ordering processes samples in coordinate order (x, y, z), meaning all samples of one band are processed before proceeding to the next. BI ordering processes samples frame by frame. The frame interleaving depth M dictates the processing order within the frame. The standard defines two special cases: Band Interleaved by Line (BIL) and Band Interleaved by Pixel (BIP), with interleaving depths equal to 1 and N_z respectively. BIL processes

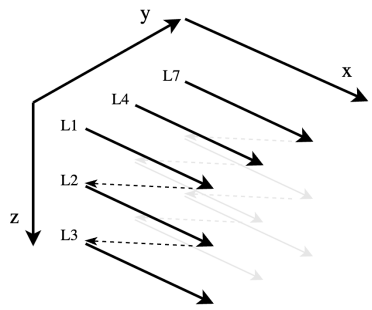

Figure 2.3: BIL processing order. Lines, L1 to L3, within the first frame are processed before proceeding to the next frame.

samples within a frame line by line in coordinate order (x, z, y), as presented by **??**. BIP processes samples in the order (z, x, y).

The following sections give a brief technical overview of the predictor and encoder stages of the issue 2 algorithm. Symbols are mentioned in text, and are the same as those used in the issue 2 standard unless otherwise is specified. As a quick reference, and for the readers convenience, all symbols are listed in Table 2.1. For a detailed review of the standard please see The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

Table 2.1: Issue 2 compressor symbol table.

Symbol	Name
$s_z(t)$	Image sample
$\sigma_z(t)$	Local sum
$\mathbf{U}_z(t)$	Local difference vector
$d_z(t)$	Central local difference
$\hat{d}_z(t)$	Predicted central local difference
$\check{s}_z(t)$	High-resolution predicted sample
$\tilde{s}_z(t)$	Double-resolution predicted sample
$\hat{s}_z(t)$	Predicted sample
$\Delta_z(t)$	Prediction residual
$m_z(t)$	Maximum error
$q_z(t)$	Signed quantizer index
$s'_z(t)$	Clipped quantizer bin center
$s''_z(t)$	Sample representative
$\theta_z(t)$	Predicted sample and sample endpoint difference
$\delta_z(t)$	Mapped quantizer index
$e_z(t)$	Double-resolution prediction error
$\mathbf{W}_z(t)$	Weight vector
$\tilde{\Sigma}_z(t)$	High-resolution accumulator
$\Gamma(t)$	Counter

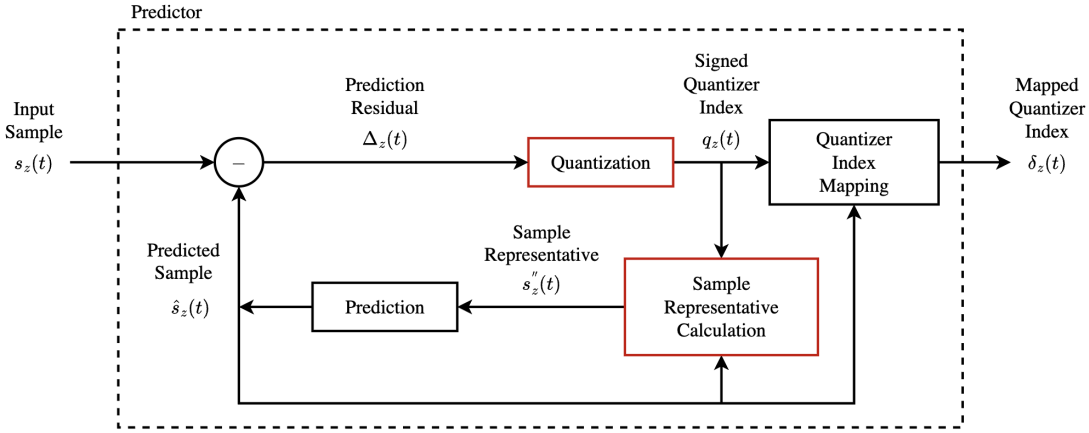

Figure 2.4: Block diagram of the predictor stage of issue 2. The quantization and sample representative steps, marked in red, are new inclusions compared to Issue 1.

2.1.1 Predictor

Figure 2.4 shows a block diagram of the issue predictor stage with new features as compared to issue 1 marked in red. The quantization step constitutes the lossy part of the algorithm.

$$\sigma_{z,y,x} = \begin{cases} s''_{z,y-1,x-1} + 2s''_{z,y-1,x} + s''_{z,y-1,x+1}, & y > 0,\ 0 < x < N_X - 1, \\ 4s''_{z-1,y,x-1}, & y = 0,\ x > 0,\ z > 0, \\ 2\left(s''_{z,y-1,x} + s''_{z,y-1,x+1}\right), & y > 0,\ x = 0, \\ 2\left(s''_{z,y-1,x-1} + s''_{z,y-1,x}\right), & y > 0,\ x = N_X - 1, \\ 4s_{\text{mid}}, & y = 0,\ x = 0,\ z = 0. \end{cases} \tag{2.3}$$

$$\mathbf{U}_z(t) = \begin{bmatrix} d_{z-1}(t) \\ d_{z-2}(t) \\ \vdots \\ d_{z-P_z^*}(t) \end{bmatrix}, \tag{2.4}$$

$$P_z^* = \min(z, P), \tag{2.5}$$

$$\hat{d}_z(t) = \mathbf{W}_z(t) \cdot \mathbf{U}_z(t). \tag{2.6}$$

$$\mathbf{W}_z(t) = \begin{bmatrix} \omega_z^{(1)}(t) \\ \omega_z^{(2)}(t) \\ \vdots \\ \omega_z^{(P_z^*)}(t) \end{bmatrix}, \tag{2.7}$$

$$\omega_z^{(1)}(1) = \frac{7}{8}2^{\Omega}, \qquad \omega_z^{(i)}(1) = \left\lfloor \frac{1}{8}\,\omega_z^{(i-1)}(1) \right\rfloor, \quad i = 2, 3, \ldots, P_z^*, \tag{2.8}$$

$$\omega_{\min} = -2^{\Omega+2},\ \omega_{\max} = -2^{\Omega+2} - 1. \tag{2.9}$$

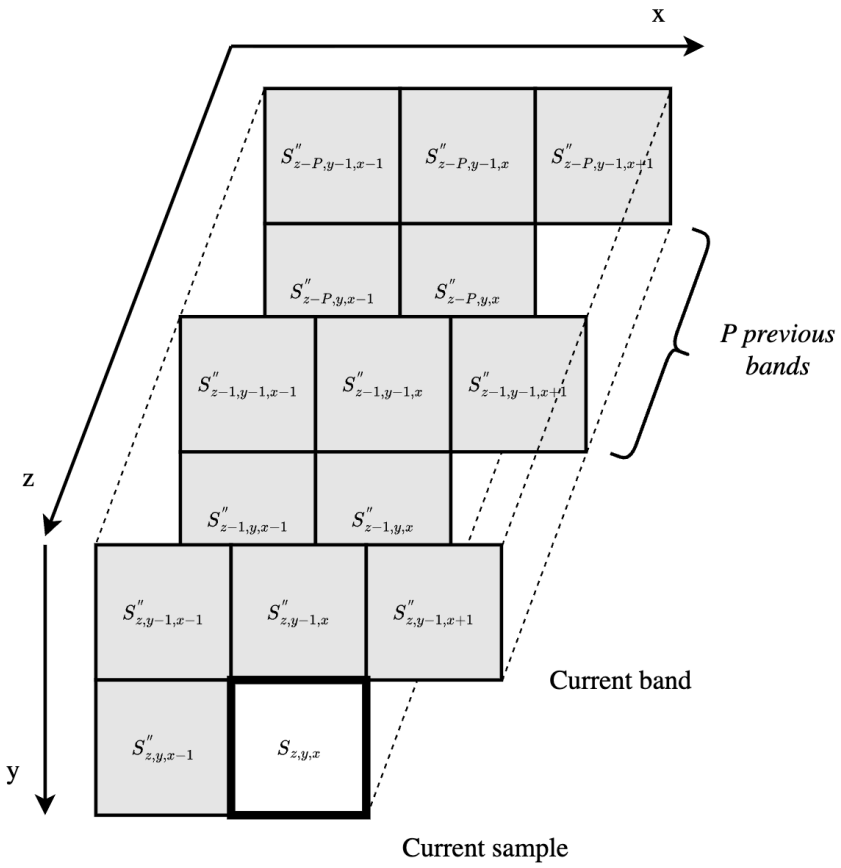

Figure 2.5: A typical prediction neighborhood. Local sums are used to find local differences in P previous bands (prediction bands) which are used to find the predicted sample.

$$\omega^{(i)}_{z,\text{offset}}(t) = \left\lfloor \frac{1}{2}\left(2^{-\rho(t)} \cdot d_{z-i}(t) \cdot \text{sgn}^{+}[e_z(t)] + 1\right) \right\rfloor, \tag{2.10}$$

$$e_z(t) = 2s'_z(t) - \tilde{s}_z(t). \tag{2.11}$$

$$\rho(t) = \text{clip}\left(\nu_{\min} + \left\lfloor \frac{t - N_x}{t_{\text{inc}}} \right\rfloor, \{\nu_{\min}, \nu_{\max}\} \right) + D - \Omega, \tag{2.12}$$

$$\omega^{(i)}_z(t+1) = \text{clip}\left(\omega^{(i)}_z(t) + \omega^{(i)}_{z,\text{offset}}(t), \{\omega_{min}, \omega_{max}\} \right), \tag{2.13}$$

Decompression

2.1.2 Header

2.1.3 Entropy Encoder

Hybrid Encoder

$$\tilde{\Sigma}_z(t) = \begin{cases} \tilde{\Sigma}_z(t-1) + 4\delta_z(t), & \Gamma(t-1) < 2^{\gamma^*} - 1, \\ \left\lfloor \dfrac{\tilde{\Sigma}_z(t-1) + 4\delta_z(t) + 1}{2} \right\rfloor, & \Gamma(t-1) = 2^{\gamma^*} - 1, \end{cases} \tag{2.14}$$

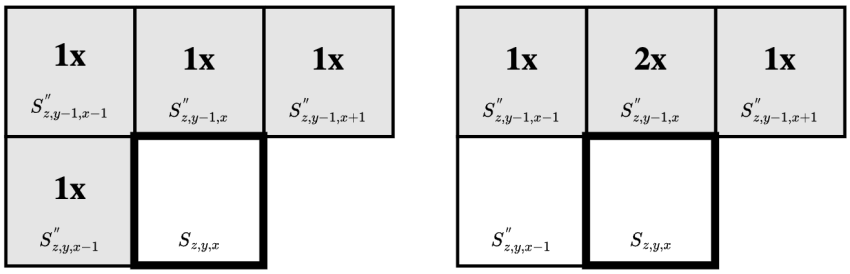

Figure 2.6: Wide and narrow neighbor-oriented local sum configurations.

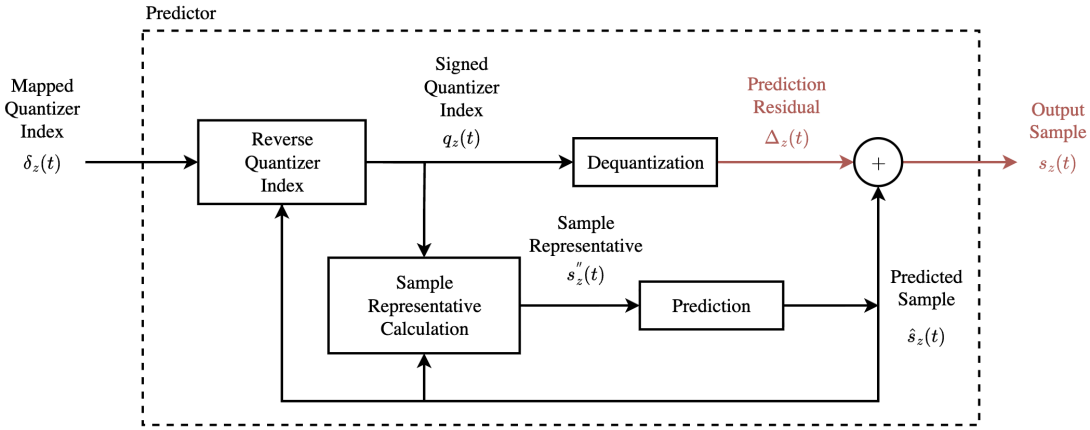

Figure 2.7: Block diagram of the predictor stage of the decompressor. The prediction model is identical to that of the compressor, except for the prediction residual and reconstructed sample during near-lossless mode, marked in red.

$$\Gamma(t) = \begin{cases} \Gamma(t-1)+1, & \Gamma(t-1) < 2^{\gamma^*}-1, \\ \left\lfloor \dfrac{\Gamma(t-1)+1}{2} \right\rfloor, & \Gamma(t-1) = 2^{\gamma^*}-1, \end{cases} \tag{2.15}$$

Decoding

2.2 Hardware Accelerators

2.3 Coding Performance and Decompressed Image Quality

$$\mathrm{CR} = \frac{N_x \cdot N_y \cdot N_z \cdot D}{\text{compressed data size (bits)}}. \tag{2.16}$$

$$\mathrm{PSNR}_z = 10 \cdot \log_{10}\left(\frac{\mathrm{MAX}_I^2}{\mathrm{MSE}_z}\right), \tag{2.17}$$

$$\mathrm{MSE}_z = \frac{1}{N_x N_y} \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \left[I_z(i,j) - K_z(i,j)\right]^2. \tag{2.18}$$

$$\mathrm{PSNR}_{HSI} = \frac{1}{N_z} \sum_{z=0}^{N_z-1} \mathrm{PSNR}_z. \tag{2.19}$$

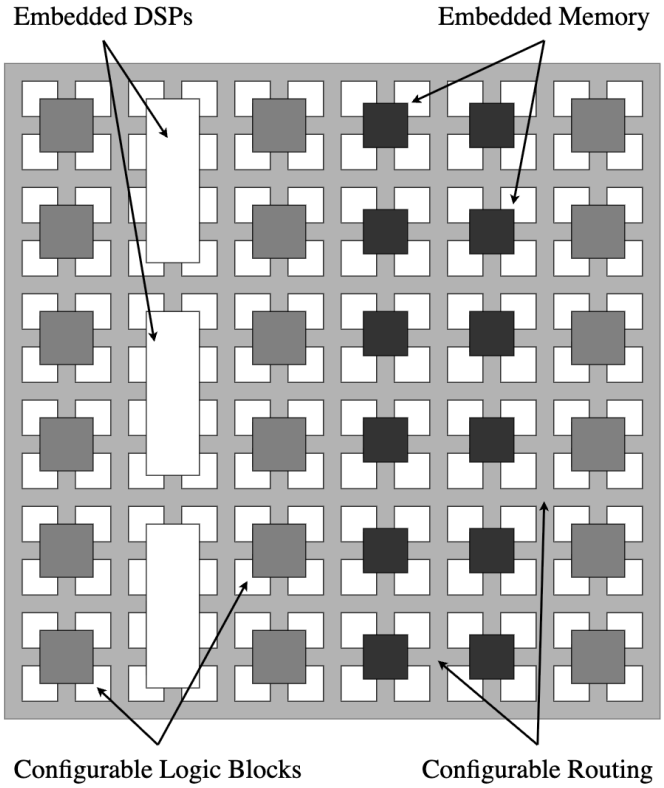

Figure 2.8: General layout of an FPGA. CLBs, DSPs, and memory blocks are placed in a configurable routing network.

Chapter 3

State-of-the-Art

Table 3.1: The number of processed samples interleaving $z-1$ and $t-1$ dependencies, including the novel FID ordering in addition to the standard orderings.

	BSQ	**BIP**	**BIL**	**FID**
$z-1$	$N_x \cdot N_y$	1	N_x	$N_z - 1$
$t-1$	1	N_z	1	N_z

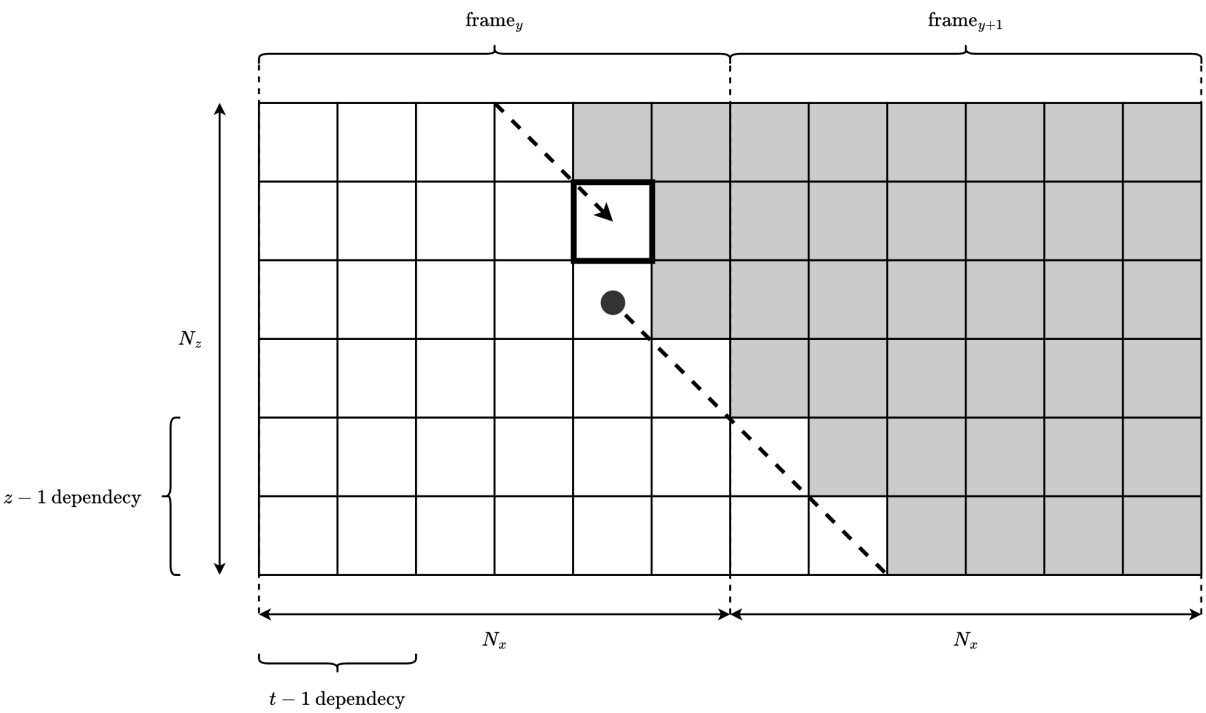

Figure 3.1

$$\operatorname{sgn}^+\big[e_z(t)\big] = \begin{cases} -1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is odd,} \\ 1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is even,} \\ \operatorname{sgn}^+\big[s_z(t) - \hat{s}_z(t)\big], & \text{otherwise.} \end{cases} \tag{3.1}$$

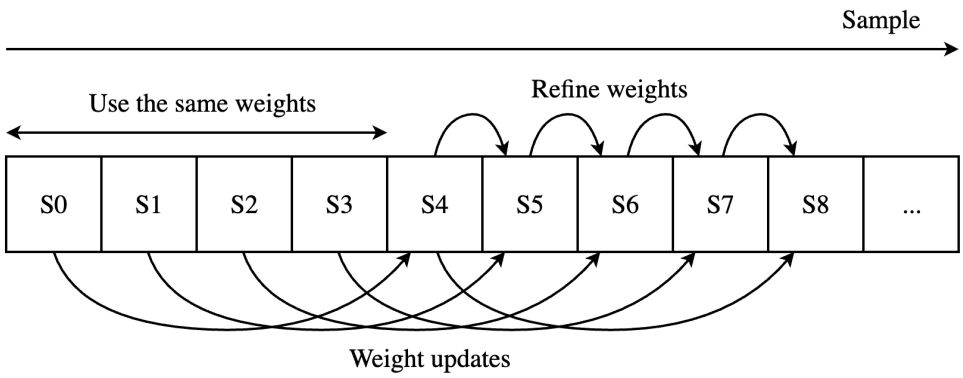

Figure 3.2: The proposed weight update scheme by Jia et al. Weights are kept unchanged for the first samples of every line, after which the sample from four samples earlier is used in the weight update calculation.

$$\omega^{(i)}_{z,\text{unclipped}}(t+1) = \begin{cases} \omega^{(i)}_{z}(t), & x \leq 2, \\ \omega^{(i)}_{z}(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3), & x = 3, \\ \left\lfloor \frac{1}{2}\left(\omega^{(i)}_{z}(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3) + \omega^{(i)}_{z}(t)\right)\right\rfloor, & \text{otherwise.} \end{cases} \tag{3.2}$$

Chapter 4

Previous Work

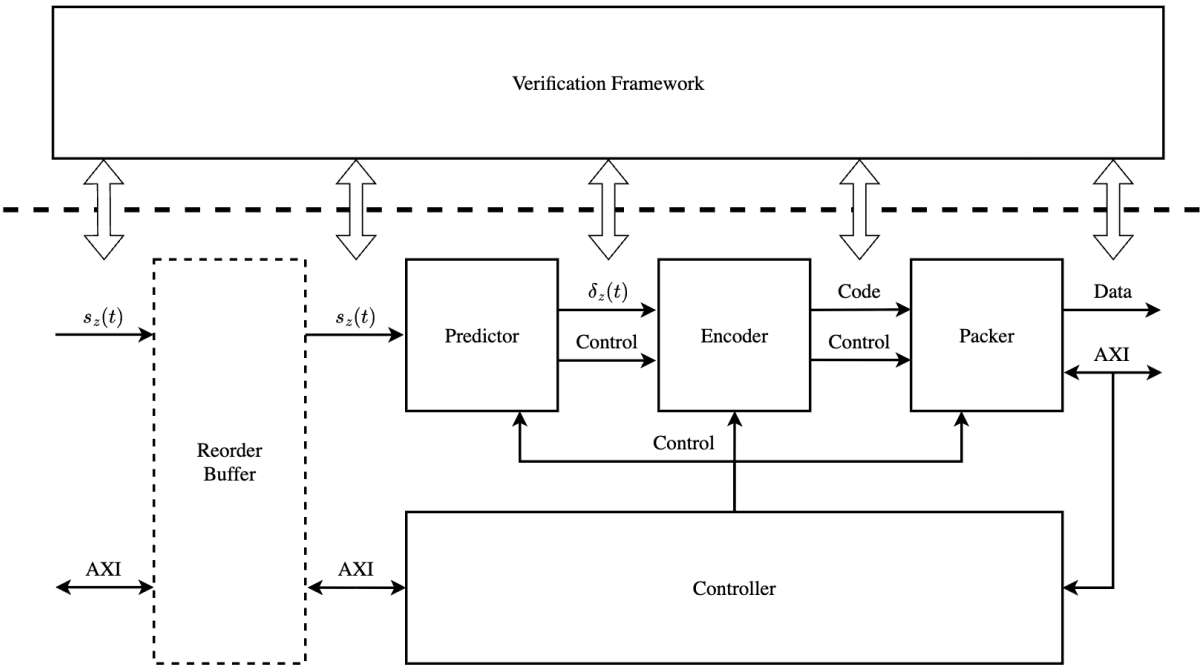

Figure 4.1: Block diagram of Vorhaug. For easy integration into HYPSO, a reorder buffer reorders samples from BIP to BIL.

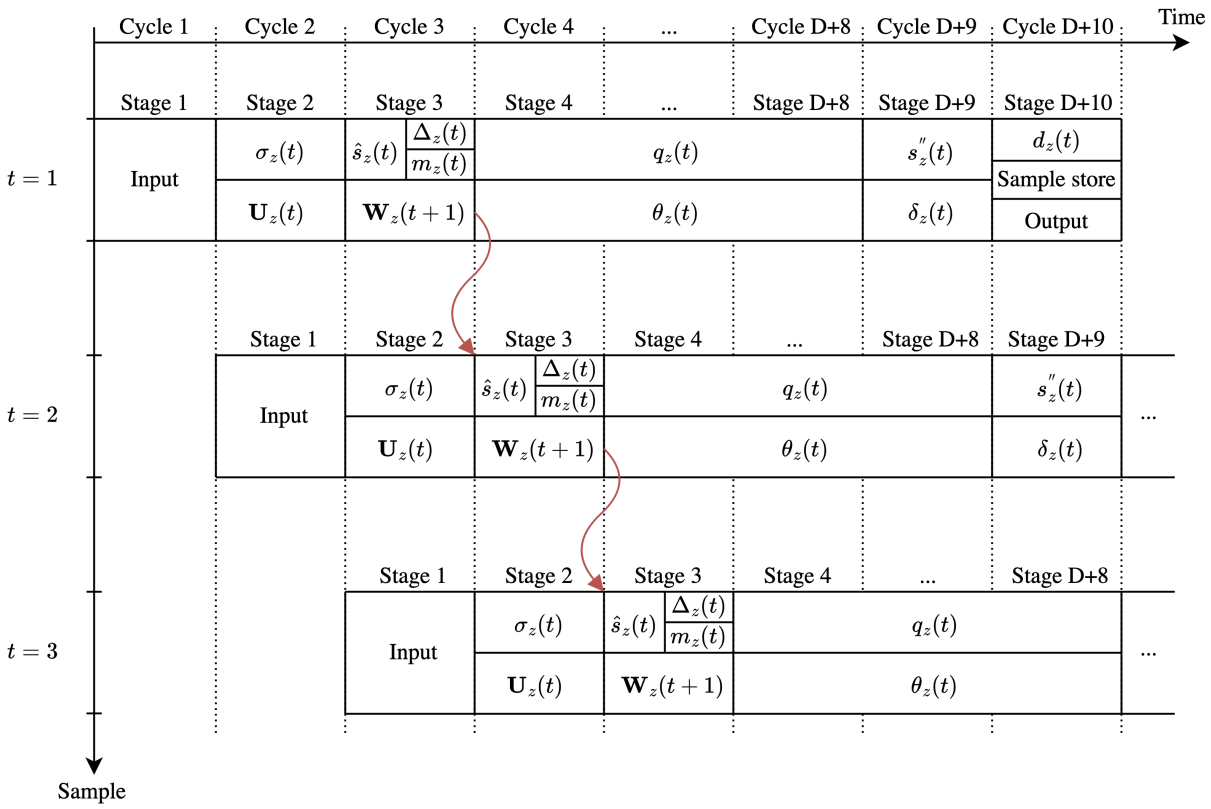

Figure 4.2: Full pipeline of Vorhaug. The indivisible weight update calculation in stage 3 restricts the throughput.

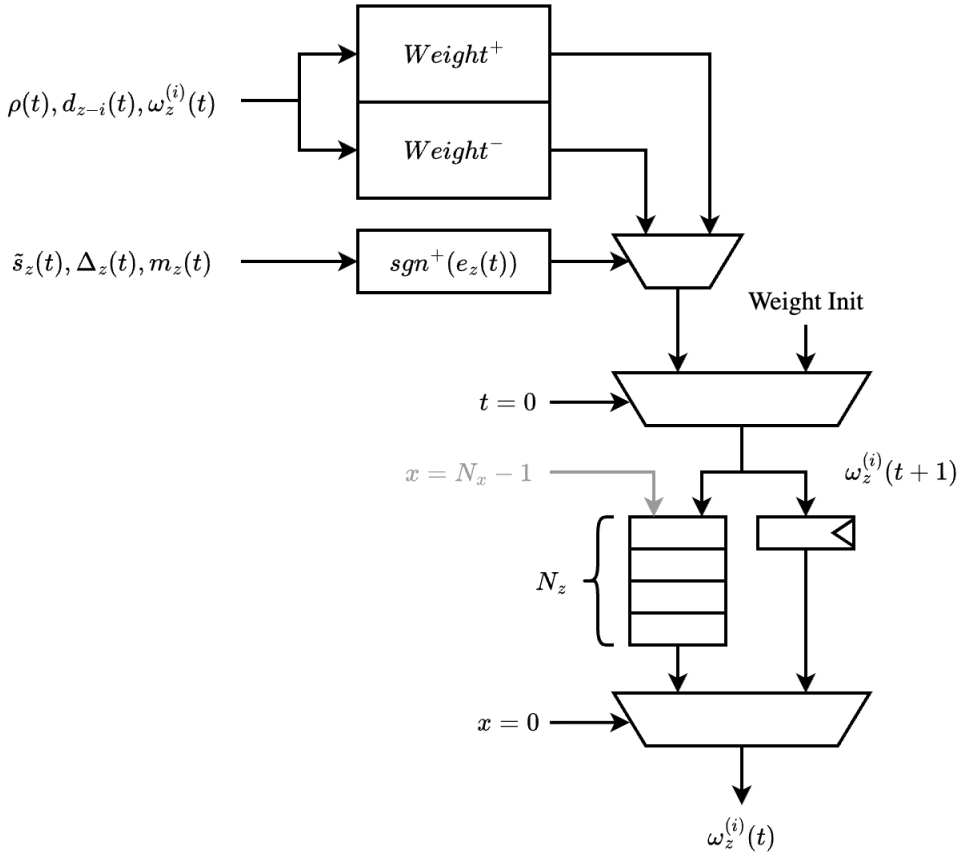

Figure 4.3: Weight update stage of Vorhaug following the mathematical optimization approach proposed by Sánchez et al. Weight updates are calculated speculatively while the sign of the prediction error resolves.

Chapter 5

Implementation and Verification

5.1 Verification Model

Predictor with compression and decompression implementation in C++.

Reconstruction of mapped quantizer indexes from compressed bitstream, hybrid encoder.

5.2 Hardware Modifications

5.3 Hardware Verification

5.4 Hardware Testing

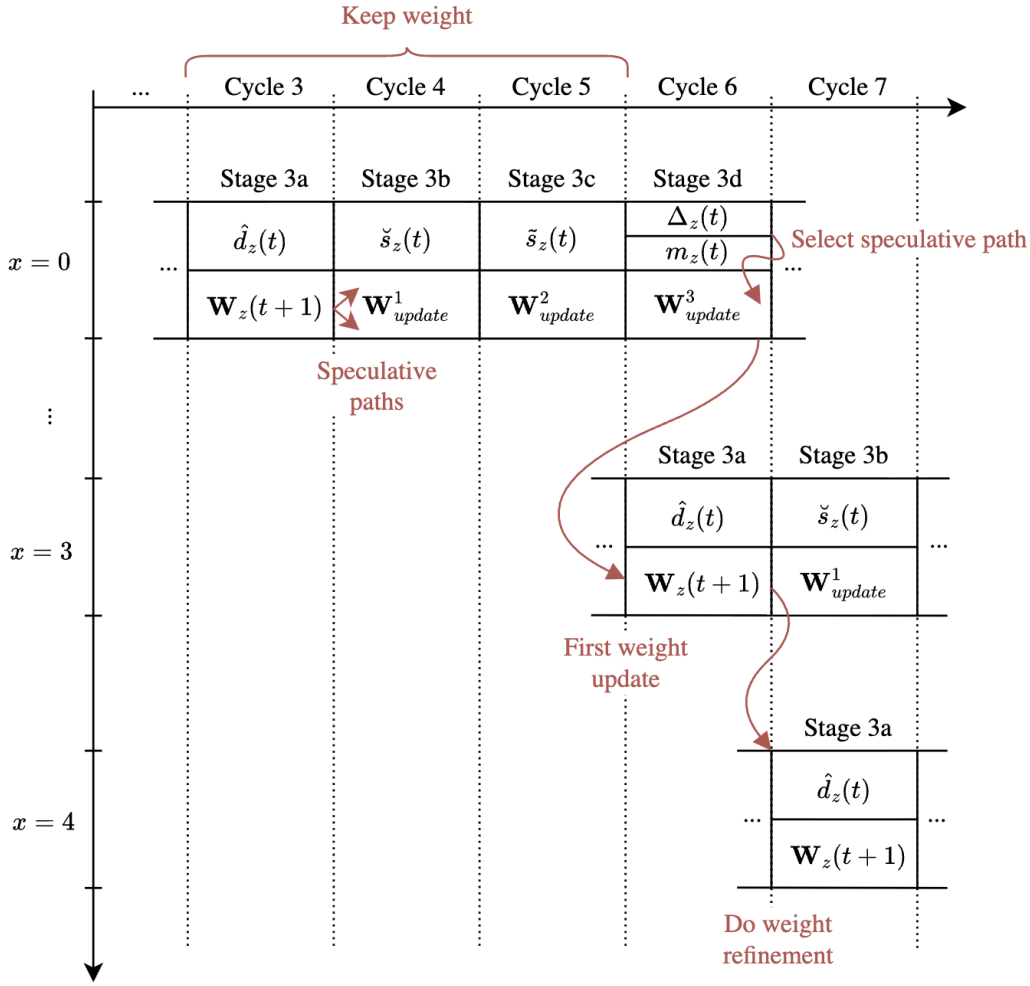

Figure 5.1: Proposed pipeline for delayed weight updates.

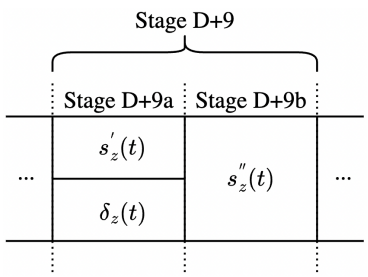

Figure 5.2: Sample representative stage split into two stages $D+9a$ and $D+9b$, reducing the new critical path after splitting the weight update stage.

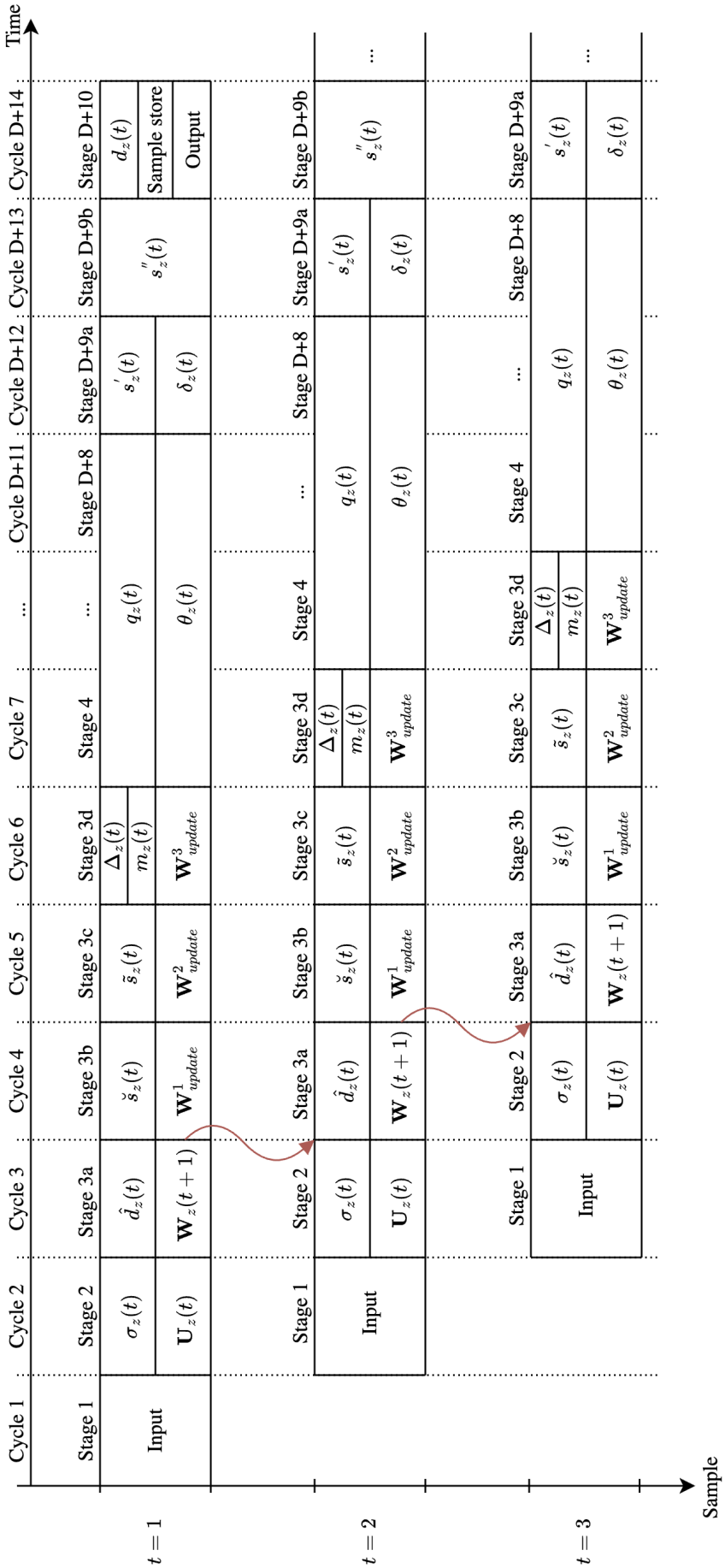

Figure 5.3: Proposed pipeline with delayed weight updates allowing splitting stage 3 into four separate stages, 3a to 3d. Additionally, the sample representative calculation in stage $D+9$ is split into two stages $D+9a$ and $D+9b$.

Table 5.1: Selected predictor and hybrid encoder parameters. Values are selected based on the work by Blanes et al., and are the same as those used by Vorhaug [5, 13]. Table recreated from [5].

Name	**Symbol**	**Configuration**
Sample type		Unsigned
Dynamic range	D	16
Sample encoding order		BIL
Entropy coder type		Hybrid coder
Quantizer method		Absolute
Number of prediction bands	P	3
Register size	R	64
Local sum type		Narrow column-oriented
Prediction mode		Reduced
Weight component resolution	Ω	19
Weight initialization method		Default
Weight update initial parameter	$\nu_{\min}$	-1
Weight update final parameter	$\nu_{\max}$	4
Weight update change interval	t_{inc}	2^6
Weight update exponent offsets	$\zeta_z^{(i)}, \zeta_z^*$	0
Periodic error limit update		Disabled
Band-varying absolute limits		Disabled
Absolute error limit	a_z	*Varied*
Band-varying relative limits		Disabled
Relative error limit	r_z	0
Sample representative resolution	Θ	3
Sample representative offset	ψ_z	7
Band-varying offset		Disabled
Sample representative damping	θ_z	3
Band-varying damping		Disabled
Unary length limit	$U_{\max}$	18
Rescaling counter size	γ^*	6
Initial count exponent	γ_0	1
Accumulator initialization value	$\tilde{\Sigma}_z(0)$	$4 \cdot 2^{\gamma_0}$

Chapter 6

Results

6.1 Verification Model

Table 6.1: Compression performance, execution time of predictor in parenthesis (TODO: find better way of displaying than in parenthesis)

Image	Predictor	Samples	Time (s)		Throughput (MS/s)	Memory (MB)
Hypso 2	C++	78361920	821	(293)	0.095	32415
Aviris	C++	77643776	1166	(290)	0.066	32091
Landsat	C++	6291456	103	(22)	0.061	2669
Hypso 2	Python	78361920	2818	(2169)	0.027	38996
Landsat	Python	6291456	263	(161)	0.024	3204

Table 6.2: Decompression

Image	Samples	Time (s)	Throughput (MS/s)	Memory (MB)
Hypso 2	78361920	1093	0.072	32414
Aviris	77643776	1834	0.042	32091
Landsat	6291456	149	0.042	2667

6.2 Compression Performance

Table 6.3: Delayed weight updates, relative 16 and absolute 4

Image	Delayed	Ordinary	Decrease
Hypso 2	2.47	2.49	−0.80%
Aviris	4.31	4.41	−2.23%
Landsat	4.82	5.21	−7.49%

Table 6.4: Reductions in compression performance for coastline image for this work and the work by Jia et al.

PAE	Coastline, Jia et al.	Coastline, this work
0	−1.53%	−0.50%
1	−2.88%	−0.78%
3	−3.86%	−0.63%
7	−4.30%	−0.98%
15	−1.23%	−1.10%

Table 6.5: PSNR of decompressed images with delayed weight updates. Difference compared to without delayed updates are degligable, exact same when rounded to two decimals.

PAE	CR	PSNR (dB)
0	2.02	N/A
1	2.53	225.86
3	3.15	207.94
7	4.03	192.54
15	5.39	178.10

6.3 RTL Results

Table 6.6

FPGA	**LUT**	**FF**	**DSP**	**BRAM**	**Power**	**Frequency**
Zynq-7030	5958	3425	4	83.5	152 mW	90.9 MHz
Zynq ZU7EV	6331	3442	4	79.5	273 mW	200.0 MHz
Zynq-7030 (predictor)	2841	2212	4	69.5	263 mW	125.0 MHz
Zynq ZU7EV (predictor)	2969	2185	4	65.5	467 mW	312.5 MHz

6.4 Hardware Testing

Figure 6.1: Decoded Hypso 2 image

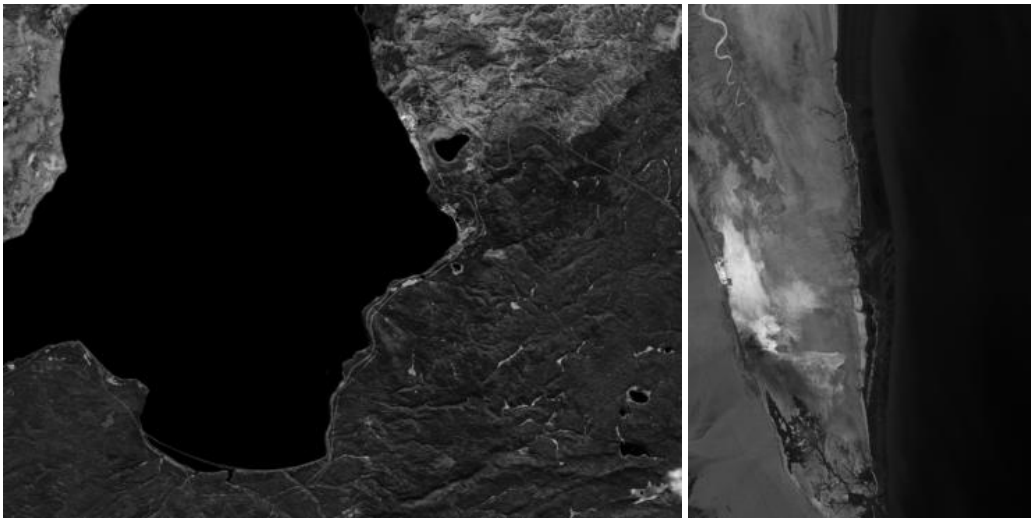

Figure 6.2: Decoded Aviris and Landsat images.

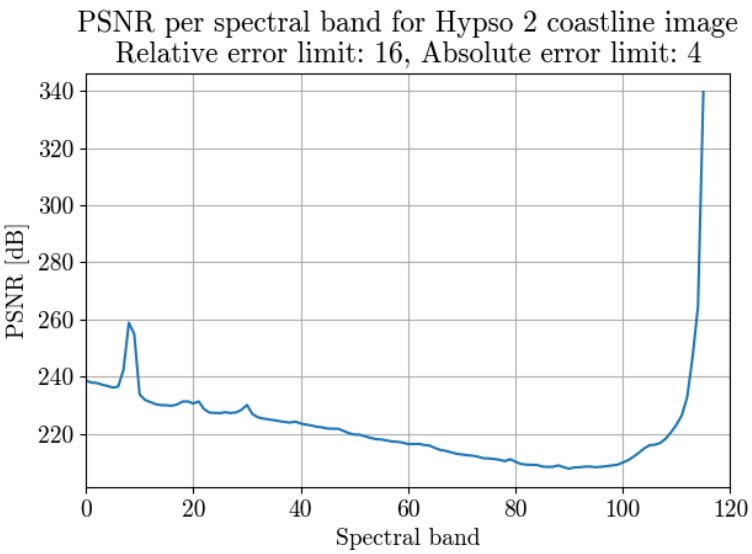

Figure 6.3

Chapter 7

Discussion and Further Work

Work should be done to properly integrate the issue 2 implementation into the HYPSO-2 on-board-processing (OPU) system. This includes writing software for header, and selection of parameters. As of now, HYPSO-2 still uses the outdated issue 1 implementation. Although fast, it cannot achieve compression rates close to those of issue 2. Issue 2 for HYPSO-3!!

Chapter 8

Conclusions

Bibliography

[1] S. Bakken, M. B. Henriksen, R. Birkeland, D. D. Langer, A. E. Oudijk, S. Berg, Y. Pursley, J. L. Garrett, F. Gran-Jansen, E. Honoré-Livermore, M. E. Grøtte, B. A. Kristiansen, M. Orlandic, P. Gader, A. J. Sørensen, F. Sigernes, G. Johnsen and T. A. Johansen, 'HYPSO-1 CubeSat: First Images and In-Orbit Characterization,' *Remote Sensing*, vol. 15, no. 3, Jan. 2023, ISSN: 2072-4292. DOI: 10.3390/rs15030755.

[2] E. J. Ientilucci and S. Adler-Golden, 'Atmospheric Compensation of Hyperspectral Data: An Overview and Review of In-Scene and Physics-Based Approaches,' *IEEE Geoscience and Remote Sensing Magazine*, vol. 7, no. 2, Jun. 2019, ISSN: 2168-6831. DOI: 10.1109/MGRS.2019.2904706.

[3] M. E. Grøtte, R. Birkeland, E. Honoré-Livermore, S. Bakken, J. L. Garrett, E. F. Prentice, F. Sigernes, M. Orlandić, J. T. Gravdahl and T. A. Johansen, 'Ocean Color Hyperspectral Remote Sensing With High Resolution and Low Latency—The HYPSO-1 CubeSat Mission,' *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, 2022, ISSN: 1558-0644. DOI: 10.1109/TGRS.2021.3080175.

[4] L. Jia, Q. Wang, L. Zhang, C. Song and P. Zhang, 'Removal of the Feedback Loop in CCSDS 123.0-B-2 during Hardware Implementation,' *IEEE Geoscience and Remote Sensing Letters*, 2025, ISSN: 1545-598X, 1558-0571. DOI: 10.1109/LGRS.2025.3601194.

[5] D. Vorhaug, 'Hyperspectral Image Compression Accelerator On FPGA Using CCSDS 123.0-B-2,' M.S. thesis, NTNU, 2024.

[6] J. Fjeldtvedt, M. Orlandić and T. A. Johansen, 'An Efficient Real-Time FPGA Implementation of the CCSDS-123 Compression Standard for Hyperspectral Images,' *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 11, no. 10, Oct. 2018, ISSN: 2151-1535. DOI: 10.1109/JSTARS.2018.2869697.

[7] The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

[8] M. Klimesh, 'Low-complexity lossless compression of hyperspectral imagery via adaptive filtering,' *Interplanetary Network Progress Report*, Jan. 2005.

[9] The Consultative Committee for Space Data Systems, 'Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Green Book Report CCSDS 120.2-G-2, 2022.

[10] D. Keymeulen, S. Shin, J. Riddley, M. Klimesh, A. Kiely, E. Liggett, P. Sullivan, M. Bernas, H. Ghossemi, G. Flesch, M. Cheng, S. Dolinar, D. Dolman, K. Roth, C. Holyoake, K. Crocker and A. Smith, 'High performance space computing with system-on-chip instrument avionics for space-based next generation imaging spectrometers (ngis),' Aug. 2018, pp. 33–36. DOI: 10.1109/AHS.2018.8541473.

[11] A. Gersho, 'Adaptive filtering with binary reinforcement,' *IEEE Transactions on Information Theory*, vol. 30, no. 2, pp. 191–199, Mar. 1984, ISSN: 1557-9654. DOI: 10.1109/TIT.1984.1056890. Accessed: 5th Dec. 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1056890.

[12] A. Sánchez, I. Blanes, Y. Barrios, M. Hernández-Cabronero, J. Bartrina-Rapesta, J. Serra-Sagristà and R. Sarmiento, 'Reducing Data Dependencies in the Feedback Loop of the CCSDS 123.0-B-2 Predictor,' *IEEE Geoscience and Remote Sensing Letters*, vol. 19, 2022, ISSN: 1558-0571. DOI: 10.1109/LGRS.2022.3213975.

[13] I. Blanes, A. Kiely, M. Hernández-Cabronero and J. Serra-Sagristà, 'Performance Impact of Parameter Tuning on the CCSDS-123.0-B-2 Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression Standard,' *Remote Sensing*, vol. 11, no. 11, Jan. 2019, ISSN: 2072-4292. DOI: 10.3390/rs11111390.

Appendix A

Block Design

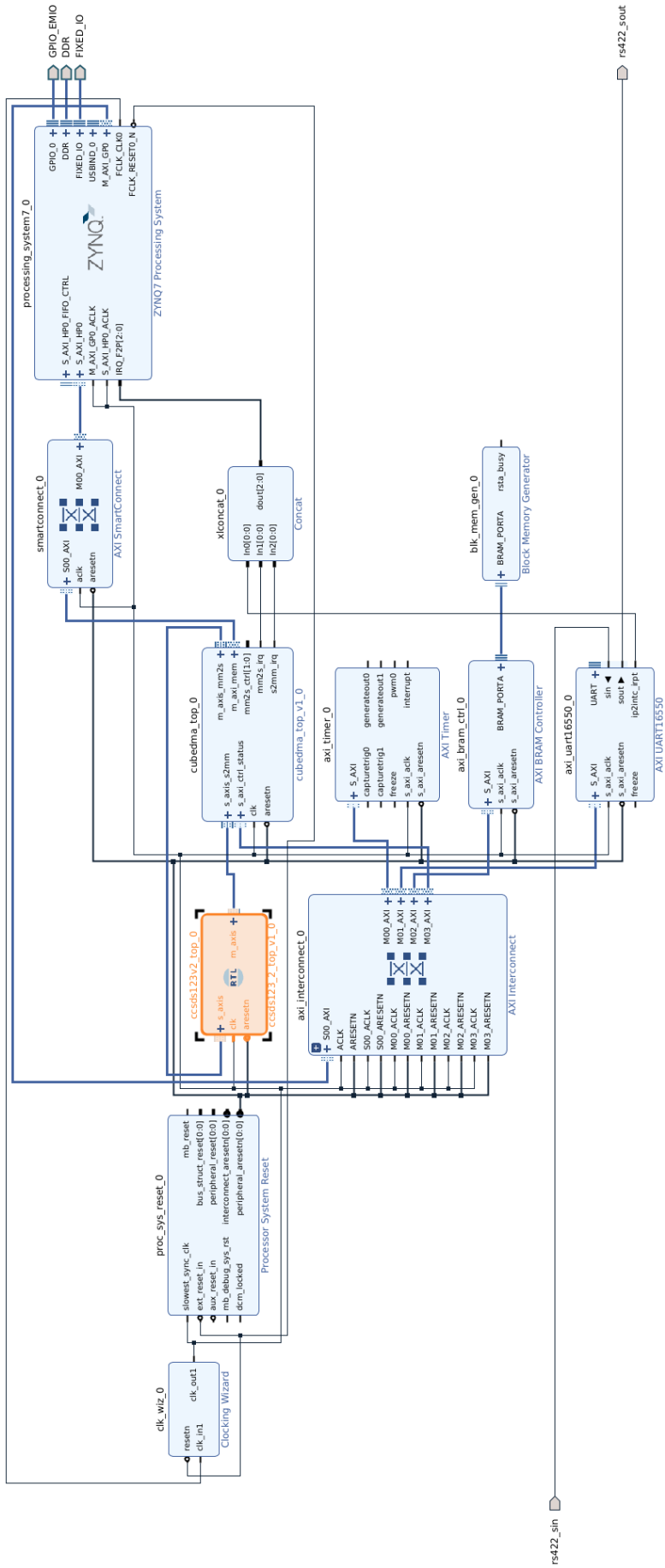
GPIO_EMIO
DDR
FIXED_IO
processing_system7_0
GPIO_0
DDR
FIXED_IO
USBIND_0
M_AXI_GP0
FCLK_CLK0
FCLK_RESET0_N
S_AXI_HP0_FIFO_CTRL
S_AXI_HP0
M_AXI_GP0_ACLK
S_AXI_HP0_ACLK
IRQ_F2P[2:0]
ZYNQ7 Processing System
smartconnect_0
S00_AXI
aclk
aresetn
M00_AXI
AXI SmartConnect
xlconcat_0
In0[0:0]
In1[0:0]
In2[0:0]
dout[2:0]
Concat
blk_mem_gen_0
BRAM_PORTA
rsta_busy
Block Memory Generator
cubedma_top_0
m_axis_mm2s
m_axi_mem
mm2s_ctrl[1:0]
mm2s_irq
s2mm_irq
s_axis_s2mm
s_axi_ctrl_status
clk
aresetn
cubedma_top_v1_0
axi_timer_0
S_AXI
capturetrig0
capturetrig1
freeze
generateout0
generateout1
pwm0
interrupt
s_axi_aclk
s_axi_aresetn
AXI Timer
axi_bram_ctrl_0
S_AXI
s_axi_aclk
s_axi_aresetn
BRAM_PORTA
AXI BRAM Controller
axi_uart16550_0
S_AXI
s_axi_aclk
s_axi_aresetn
freeze
UART
sin
sout
ip2intc_irpt
AXI UART16550
rs422_sout
ccsds123v2_top_0
s_axis
clk
aresetn
RTL
m_axis
ccsds123_2_top_v1_0
axi_interconnect_0
S00_AXI
ACLK
ARESETN
S00_ACLK
S00_ARESETN
M00_ACLK
M00_ARESETN
M01_ACLK
M01_ARESETN
M02_ACLK
M02_ARESETN
M03_ACLK
M03_ARESETN
M00_AXI
M01_AXI
M02_AXI
M03_AXI
AXI Interconnect
proc_sys_reset_0
slowest_sync_clk
ext_reset_in
aux_reset_in
mb_debug_sys_rst
dcm_locked
mb_reset
bus_struct_reset[0:0]
peripheral_reset[0:0]
interconnect_aresetn[0:0]
peripheral_aresetn[0:0]
Processor System Reset
clk_wiz_0
resetn
clk_in1
clk_out1
Clocking Wizard
rs422_sin

Figure A.1